

Chapter 11: REST API Deep Dive & Documentation

Welcome to Chapter 11! Backend developers apna mostly time APIs design aur debug karne mein nikalte hain. Ek interviewer check karna chahta hai ki aapko HTTP Methods, Status Codes, aur JSON payloads ki kitni deep understanding hai.

Is chapter mein hum pehle ek Master Table dekhenge, aur phir sabse important APIs ka operation (Flow, Validation, DB Query) detail mein samjhenge.

 1. API Master Documentation Table

Method	Endpoint	Purpose	Auth Required	Validation (Zod)	Status Codes
POST	/api/auth/register	Register new user	✗ No	email, password, name	201, 400, 409
POST	/api/auth/login	Login user, issue JWT	✗ No	email, password	200, 400, 401
POST	/api/auth/logout	Clear HttpOnly Cookie	✓ Yes	None	200

GET	/api/auth/me	Get logged-in user profile	✓ Yes	None	200, 401, 404
POST	/api/notes	Upload & Process PDF	✓ Yes	multipart/form-data	201, 400, 500
GET	/api/notes	List user's PDFs	✓ Yes	Pagination (page, limit)	200
POST	/api/chat	Create new chat session	✓ Yes	notelds, title	201, 400, 403
POST	/api/chat/message	Send query to AI (RAG)	✓ Yes	chatId, query	200, 400, 404
POST	/api/quizzes/generate	Generate AI Quiz	✓ Yes	noteld, count, diff.	201, 400, 500
POST	/api/quizzes/:id/submit	Grade user answers	✓ Yes	answers map	200, 400
GET	/api/dashboard	Get real-time stats	✓ Yes	None	200



2. Deep Dive: The Critical Endpoints

Here we break down exactly what happens when the frontend hits these specific routes.

Endpoint A: User Login (POST /api/auth/login)

- **Method & Route:** POST /api/auth/login
 - **Purpose:** User ke credentials verify karna aur session tokens dena.
 - **Request Body:** { "email": "user@test.com", "password": "Password123!" }
 - **Validation:** Zod middleware check karta hai ki email valid format mein hai, aur password empty nahi hai.
 - **Authentication:** Public route (Bina JWT ke access kar sakte hain).
 - **Database Query:** `User.findOne({ email: '...', passwordHash: '...' }).select('+passwordHash')`
 - **Business Logic:**
 1. User dhoondo.
 2. `bcrypt.compare()` se password match karo.
 3. Agar sahi hai toh JWT Access aur Refresh tokens banao.
 - **Response:**
 - **Headers:** `Set-Cookie: refreshToken=...; HttpOnly;`
 - **Body (JSON):** { "status": "success", "data": { "accessToken": "...", "user": { ... } } }
 - **Error Handling:** Agar password galat hai toh 401 Unauthorized. Agar server crash ho jaye toh 500 Internal Server Error via `catchAsync`.
 - **Flow:** UI ⇒ Router ⇒ Auth Rate Limiter ⇒ Validation Middleware ⇒ Auth Controller ⇒ Auth Service ⇒ Mongoose ⇒ Response.
-

Endpoint B: Upload PDF (POST /api/notes)

- **Method & Route:** POST /api/notes
- **Purpose:** File receive karna aur AI parsing shuru karna.
- **Request Body:** `multipart/form-data` (A file object attached to the request).
- **Validation:** Multer middleware check karta hai `mimetype === 'application/pdf'` aur `size < 10MB`.

- **Authentication:** `protect` middleware ensure karta hai ki request ke header mein valid JWT hai.
 - **Database Query:** `Note.create({ ownerId: req.user._id, title: "Biology", status: "processing" })`
 - **Business Logic & Flow:**
 1. Multer file ko disk par save karta hai.
 2. Controller turant MongoDB mein note "processing" banata hai aur Frontend ko 201 Created bhejta hai.
 3. Controller background mein `pdf.service.js` (OCR) -> `embeddings.service.js` (Chunks & Vectors) ko fire karta hai.
 - **Response (Immediate):** `{ "status": "success", "data": { "note": { "status": "processing" } } }`
 - **Error Handling:** Agar PDF corrupted hai ya file badi hai, Multer HTTP 400 fekega jise global error handler pakad lega.
-

Endpoint C: Chat with Document (`POST /api/chat/message`)

- **Method & Route:** `POST /api/chat/message`
- **Purpose:** RAG pipeline execute karke AI se answer lena.
- **Request Body:** `{ "chatId": "12345", "query": "What is ATP?" }`
- **Validation:** Zod checks if `chatId` is a valid MongoDB ObjectId aur query length > 0.
- **Authentication:** JWT required.
- **Database Query:**
 - `Chat.findOne({ _id: chatId, ownerId: req.user._id })` (Tenant Isolation!)
 - `Message.create({ role: 'user' })` and `Message.create({ role: 'assistant' })`
- **Business Logic & Flow:**
 1. Controller request service ko deta hai.
 2. Service Gemini se query ka vector banwati hai.
 3. Local Vector DB mein Cosine search hota hai.
 4. Prompt ban kar Groq Llama-3 ko jata hai.

5. Answer aakar DB mein as a Message save hota hai.

- **Response:** { "status": "success", "data": { "aiMessage": { "content": "ATP is energy...", "citations": [...] } } }
 - **Error Handling:** LLM Quota Limit 429 Too Many Requests ko explicitly handle karke frontend par friendly message bheja jata hai.
-

Endpoint D: Fetch Dashboard (GET /api/dashboard)

- **Method & Route:** GET /api/dashboard
- **Purpose:** User ke stats (quizzes taken, avg score) dikhana.
- **Request Body:** None (GET request).
- **Authentication:** JWT required.
- **Database Query:** Multiple `Model.aggregate()` pipelines running concurrently.
- **Business Logic & Flow:**
 - `Server Promise.all()` use karke ek sath 4 alag collections (Notes, Quizzes, Flashcards, Chats) par queries fire karta hai.
 - *Why?* Taaki 4 alag APIs banakar network overhead na badhe. Ek hi API fast result dede.
- **Response:**

```
{
  "status": "success",
  "data": {
    "pdfAnalytics": { "totalUploads": 5 },
    "quizStatistics": { "averageScore": 85 }
  }
}
```



Generated Interview Questions

Q: Ek API 'Idempotent' hone ka kya matlab hota hai? Konsi methods idempotent hoti hain? A: "Idempotent ka matlab hai ki agar aap same API ko 1 baar call karo ya 100 baar, server ki state same hi rahegi. GET, PUT, aur DELETE idempotent hote hain. Lekin POST (jaise Create Note) idempotent nahi hota, kyunki har call par ek naya note create ho jayega."

Q: Tumhare saare endpoints `/api/...` se kyun start hote hain? A: "Taaki frontend aur backend ko ek hi domain par host karte waqt Nginx ya Load Balancer ko pata ho ki kaunsi request static React files (HTML/CSS) ke liye hai aur kaunsi request Node backend ko forward karni hai."

Q: Jab user wrong password dalta hai, tum backend se kya status code bhejte ho aur kyu? A: "Main 401 `Unauthorized` bhejta hoon. 400 `Bad Request` validation ke liye hota hai (jaise email mein '@' na hona). 401 explicitly authentication failure ke liye designated hai standard REST protocols mein."

Q: Tumhari Upload API background processing kaise karti hai bina HTTP connection timeout kiye? A: "Controller pehle DB entry banata hai aur `res.status(201).json()` call karke response bhej deta hai jisse HTTP connection close ho jata hai. Lekin main Express function ke andar background async function ko await nahi karta (Fire and Forget). Node ka event loop use background mein process karta rehta hai."

⚡ Quick Revision Notes for Chapter 11

- **REST Verbs:** POST for Create, GET for Read, PATCH/PUT for Update, DELETE for Delete.
- **Auth Flow:** Public endpoints (`/login`) lack JWT protection. Private endpoints check `req.user`.
- **Validation First:** Middleware (Zod) blocks bad data *before* it hits the controller.
- **Tenant Isolation:** Every protected API query checks `{ ownerId: req.user._id }`.
- **Error Propagation:** `catchAsync` routes all errors from the controller directly to the `error.middleware.js` using

```
next(err).
```